AEROSYNAPSE AI / DOCUMENTAȚIE TEHNICĂ

Dovezi pentru date, calcule și intervenții aeroportuare

Originea valorilor, verificarea formulelor și ce susțin sursele din aeroporturi reale.

Referințe verificate: **31 octombrie 2026**
Cod analizat: 9e413cf0486b549b0c04fa47ebb08a6ff3d0353d

> **Datele demo sunt sintetice.** Rezultatele sunt calculate din ipotezele modelului. Sursele externe documentează precedente reale, dar nu certifică valorile sau îmbunătățirile simulate la Craiova.

Cuprins

1. Ce origine are fiecare categorie de date 2

2. De unde provin valorile concrete din Bottleneck 2

3. Cum se formează rezultatele și cum se verifică manual 3

4. De ce recomandarea este calculabilă, dar rămâne condiționată 4

5. Cum se interpretează procentele și prognozele 5

6. Dovezi din aeroporturi reale și legătura cu modificările propuse 6

7. Cum se verifică dovezile tehnice 7

8. Ce lipsește pentru a susține un beneficiu la un aeroport anume 8

Anexa A. Surse externe 9

Anexa B. Referințe la cod 10

> **Verificare tehnică:** 236 de teste existente au trecut în auditul local. Acest rezultat verifică implementarea selectată; nu reprezintă validare pe trafic aeroportuar real.

1. Ce origine are fiecare categorie de date

Categorie	Cum se obține în proiect	Ce dovedește și ce nu dovedește
Terminal, controale, porți, personal, trasee inițiale	Valori scrise explicit în SyntheticScenarioSeeder.cs [C01], cu IsSynthetic = true	Se pot verifica valorile configurate; nu sunt un inventar măsurat al unui aeroport
Orar, 600 pasageri, seed 42, capacitate Bottleneck	Preset în studio.models.ts [C02], funcția makeDraft	Sunt intrări alese pentru demonstrație; nu vin din rezervări sau dintr-un flux de zboruri reale
Sosiri, durate de serviciu, grupuri	generator.py [C03], generate_population	Eșantionare reproductibilă a distribuțiilor configurate; nu comportamente observate pe teren
Cozi, timpi, pasageri procesați	processes.py [C04], process	Evenimente executate în timpul virtual al simulatorului; nu telemetrie fizică
Mean, mediană, P95, debit, utilizare	metrics.py [C05], wait_statistics și KpiCollector.sample	Agregări ale observațiilor simulate, cu populație și numitor definite
Alocare personal și ture	staffing.py [C06] și shifts.py [C07]	Soluții pentru obiectivul și restricțiile modelului; fezabilitatea fizică depinde de corectitudinea intrărilor
Prognoze și rezultate ML	forecast.py [C08], dataset.py [C09], model.py [C10]	Proiecții și evaluări pe date sintetice, fără acuratețe demonstrată pe trafic aeroportuar real
Energie și cost	energy/engine.py [C11], energy	Calcul dimensional pe puteri și tarife presupuse/configurate, fără măsurare din contoare
Cifre din aeroporturile citate în secțiunea 6	Rapoarte ale operatorilor, constructorului sau cercetare bazată pe măsurători	Informație externă atribuită sursei; nu trebuie prezentată drept rezultat AeroSynapse

Un ID, un hash sau eticheta isSynthetic oferă trasabilitate în sistem. Un hash nu certifică faptul că intrarea provine dintr-un aeroport real.

2. De unde provin valorile concrete din Bottleneck

În studio.models.ts [C12]:

- BOTTLENECK_RATE = 3: rata presupusă, în pasageri/minut/unitate de serviciu.
- BOTTLENECK_LANES = 3: limita declarată a demonstrației. Nu este dedusă din modelul 3D sau dintr-un plan real.
- Cinci plecări la minutele 25, 40, 55, 70, 85, fiecare cu 120 pasageri: 5 × 120 = 600.
- Orizontul este 90 minute, seed-ul este 42, iar un singur membru de personal este alocat inițial controlului ales.
- Pasagerii merg direct la securitate în acest preset; check-in-ul nu este inclus în acest experiment.

Ferestrele de sosire sunt uniforme, între max(0, plecare − 45) și plecare − 10, conform bottleneckArrivalWindow. Rezultă [0,15], [0,30], [10,45], [25,60], [40,75] minute. În acest preset, serviciul este constant: generatorul împarte multiplicatorul capacityPerMinute / 3 la rata controlului, obținând 1/3 minut = 20 secunde. Nu confundăm acest multiplicator cu un timp de serviciu deja exprimat în minute. Vezi configurarea profilurilor [C13] și conversia în timp [C14].

De ce apare coada: chiar raportată la întregul orizont, cererea este 600 / 90 = 6,67 pasageri/min, peste capacitatea de 3. O unitate ar putea termina **cel mult 270 servicii** în 90 minute, chiar dacă ar lucra fără întrerupere de la minutul 0. Prin urmare, cel puțin 330 ar rămâne neprocesați în aceste condiții; timpul până la prima sosire poate reduce suplimentar numărul terminat. Trei unități oferă o capacitate nominală de 9 pasageri/min și un plafon de 810 servicii în 90 minute, dar suprapunerea valurilor poate produce în continuare cozi. Numărul efectiv se obține prin simulare, nu prin egalarea cu plafonul teoretic.

Limită de interpretare: codul echivalează un „operator” cu o unitate paralelă de serviciu/o linie abstractă. O linie reală implică echipamente și posturi de lucru. O capacitate publicată per linie nu dovedește aceeași capacitate per angajat. Pentru un plan real trebuie modelate separat liniile, echipele, calificările și disponibilitatea echipamentelor. Nici numărul de oameni disponibili în layout nu dovedește că există fizic trei linii instalate.

Crearea din nou a unui preset generează UUID-uri noi pentru zboruri. Generatorul folosește și UUID-ul zborului la construirea fluxului aleator; **seed 42, singur, nu garantează aceiași pasageri între două scenarii create separat**. Pentru reproducere se păstrează și toate intrările/ID-urile scenariului capturat.

3. Cum se formează rezultatele și cum se verifică manual

Populația și traseul

stream din generator.py:16 [C15] construiește fluxuri NumPy PCG64 din seed, UUID de zbor și scop (arrival, service, travel, groups). Fluxurile separate împiedică o extragere pentru serviciu să consume aleatorietatea sosirilor. quantile implementează distribuțiile constantă, uniformă, normală trunchiată, exponențială mărginită și triunghiulară. Parametrii acestor distribuții sunt ipoteze; numele unei distribuții nu constituie o calibrare pe date reale. În special, media parametrizată a unei distribuții trunchiate nu trebuie confundată cu media efectivă a eșantionului rezultat. Fără passengerGeneration, motorul checkpoint folosește un alt mod: intervale dintre sosiri și servicii exponențiale, cu fluxuri random.Random inițializate din seed; vezi processes.py:63 [C16]. Explicația distribuției trebuie să corespundă modului efectiv al run-ului.

serviceMinutes = serviciu_eșantionat × multiplicatori / serviceRatePerMinute. Cererea per minut și per sfert de oră se numără din aceeași listă de sosiri, iar capacitatea estimată este staffCount / media_duratelor_de_serviciu; vezi generator.py:133 [C17].

Cu navigație activă, NavigationGraph.search [C18] folosește Dijkstra, cu timpii travelSeconds drept cost principal, plus închideri și reguli de acces. Timpul se obține din suma timpilor muchiilor și multiplicatorii profilului, nu prin măsurarea aspectului animației 3D. Traseele și timpii lor trebuie măsurați/calibrați separat pentru un aeroport real.

Coada și KPI-urile

În processes.py:182 [C04], pasagerul intră în coadă, primește un slot FIFO, începe serviciul, așteaptă durata virtuală și abia apoi este numărat ca procesat și începe mersul. Numărul de servicii simultane este limitat. Este o implementare pe evenimente SimPy, cu sloturi FIFO proprii pentru capacitate modificabilă; documentația SimPy [S01] explică mecanismul de așteptare pentru resurse limitate.

Indicator	Formula implementată și populația
Așteptarea unui pasager	început_serviciu − sosire_la_coadă, în minute
Mean wait	Suma așteptărilor / numărul serviciilor terminate până la minutul observat
Mediană	Valoarea centrală; pentru număr par, media celor două valori centrale
P95	Așteptările ordonate, poziția ceil(0,95 × n) cu indexare de la 1; fără interpolare
Throughput în fereastră	Servicii terminate în (a,t] / (t−a), în pasageri/minut
Throughput cumulativ	Total servicii terminate / minute scurse
Utilizare	Integrala numărului de sloturi ocupate / (capacitate nominală × durata ferestrei)
Boarding late risk în motorul checkpoint	Număr cu termen ratat / populație cu termen cunoscut; este o fracție observată în simulare, nu o probabilitate ML calibrată
Reconciliere la minutul t	Populație modelată = nesosiți + în coadă + în serviciu + în mers + la poartă + fără traseu

Sursa acestor formule este metrics.py:19 [C05], metrics.py:53 [C19] și urmărirea termenului [C20]. Fără eșantion de servicii terminate, mean/P95 din kpiResult sunt null. La capacitate zero, utilizarea este null. În cazul incidentelor/turelor, utilizarea păstrează convenția capacității nominale; effectiveCapacity, când este emis, trebuie citit separat. Nu interpreta automat utilizarea ca ocupare raportată la personalul efectiv din fiecare minut. Payload-urile istorice metrics/processResult au și convenții proprii (inclusiv zero pentru unele medii fără servicii); explicația unei valori trebuie legată de câmpul și versiunea efectiv afișate.

Exemplu verificabil fără estimări statistice: kpi-request.json [C21] are doi pasageri sosiți la minutul 0, o unitate de serviciu și un minut/serviciu. Primul începe la 0 și termină la 1, al doilea începe la 1 și termină la 2. Așteptările sunt [0,1]: media și mediana sunt 0,5 minute, P95 este 1 minut. La minutul 2, utilizarea cumulativă este 2/(1×2)=100%, debitul cumulativ este 2/2=1 pasager/min. La minutul 10, ambele valori cumulative sunt 0,2, iar debitul ultimei ferestre este zero. Graful acestui exemplu are două muchii de câte 60 secunde: sosirile la poartă sunt la minutele 3 și 4; cu termen la minutul 3, unul din doi întârzie. Sosirea exact la termen este acceptată. Testul existent [C22] verifică aceste egalități.

Media calculată numai pe servicii terminate poate părea mică atunci când mulți pasageri încă așteaptă. Ea trebuie prezentată împreună cu waitSampleCount, unprocessedCount, coada și riscul de întârziere. Nici o medie mică, nici un throughput mare nu dovedesc singure o îmbunătățire pentru toată populația.

La agregarea mai multor controale se folosesc ponderile WaitSampleCount, iar P95 se recalculează din așteptările individuale reunite; nu se face media percentililor. Implementare: StaffingEvaluationReader.cs:110 [C23].

4. De ce recomandarea este calculabilă, dar rămâne condiționată

StaffingOperations.cs:78 [C24] ia cererea de vârf din populația generată și calculează capacitatea per unitate ca floor(1000 / media_serviciului), când media este pozitivă; fără observații folosește rata configurată a controlului. Cererea este exprimată tot în miimi de pasager/minut. staffing.py [C06] caută valori întregi x[c] pentru:

```
min Σc [max(0, demandMilli[c] − capacityMilli[c] × x[c])
      + changeCost[c] × abs(x[c] − currentStaff[c])]
```

Restricțiile includ totalul disponibil, bugetele terminalelor, controalele închise, minimele, maximele și physicalLimit. Limita fizică este introdusă în model, nu verificată pe teren de solver. changeCost este o penalizare de scor; nu este automat un cost în lei. Statutul optimal dovedește optimul acestui obiectiv în model, iar feasible indică doar o soluție admisibilă găsită; vezi documentația oficială CP-SAT [S02].

Exemplul staffing-request.json [C25] pornește de la alocarea [0,3], cu cereri [6000,1000], capacități [3000,1000] și penalizare 10. Alocarea [2,1] acoperă cererea, iar scorul este 0 + 10×2 + 0 + 10×2 = 40. Scorul inițial este 6000. **Scăderea scorului nu este o scădere cu același procent a timpului de așteptare.** Testele solverului [C26] verifică acest exemplu, iar testul de la linia 106 compară soluții cu enumerarea exhaustivă pe 30 de probleme mici.

Pentru ture, shifts.py [C07] include intervale de 15 minute, ture contigue, pauze, mutări și capacitate pierdută la transfer; coada se reportează între intervale. Aceste reguli sunt parametri ai modelului, nu o confirmare că disponibilitatea oamenilor sau regulile locale au fost verificate.

Candidații sunt apoi rulați în SimPy. StaffingEvaluationWorkflow.cs:66 [C27] păstrează intrările experimentului, schimbând alocarea/programul; StaffingEvaluationPolicy.Assess [C28] verifică degradările și rezultatele disponibile înainte de recomandare. Acceptarea în model nu reprezintă validare operațională externă.

5. Cum se interpretează procentele și prognozele

În interfață, percentChange [C29] calculează 100 × (candidat − baseline) / baseline. Exemplu: de la 20 la 12 minute înseamnă −40%, adică o reducere de 40%. Formula cu semnul invers exprimă reducerea ca număr pozitiv. Nu se calculează procent dacă baseline este nul sau nepozitiv. Afișarea poate rotunji secundele/minutele sau procentele; verificarea se face pe valorile brute.

RunComparisonPolicy.cs [C30] verifică run-uri distincte, rezultate valide, motor/build, layout, interval și intrările populației. Un seed comun controlează aleatorietatea numai împreună cu intrările capturate. Cu repartizare între mai multe linii, personalul poate schimba și checkpoint-ul ales de grup; păstrarea sosirilor nu înseamnă că traseele rezultate trebuie să rămână identice. Vezi open_lanes [C31] și testul repartizării [C32].

O singură pereche baseline/candidat oferă rezultatul acelui experiment. Pentru robustețe se repetă aceleași seed-uri în ambele variante, se raportează diferențele pereche, dispersia, rulările eșuate și pasagerii neprocesați. Variația între replicări sintetice nu este incertitudinea completă a unui aeroport real.

Prognoza fluidă folosește Q[t+1] = max(0, Q[t] + sosiri[t] − capacitate[t]); cu 20 în coadă, 8 sosiri și 6 servicii/minut, proiecția următoare este 22. Câmpul kind este simulated_estimate, conform forecast.py:124 [C33]. Prognoza nu reprezintă o observație viitoare. Datasetul ML se declară explicit synthetic_simpy_simulation; separarea antrenare/calibrare/test este pe scenariu, nu pe rânduri din același run. Nu atribuim aeroporturilor reale scorurile ML obținute astfel.

Pentru energie, E[kWh] = Σ(unități × putere[kW/unitate] × minute/60), iar cost[RON] = E × tarif[RON/kWh]. De exemplu, 3 unități de 1,5 kW active 90 minute dau 6,75 kWh. Valoarea implicită 1,5 kW este o ipoteză din energy/engine.py:16 [C11], nu specificația unui echipament real; fără tarif nu se produce o estimare monetară. Modelele separate de bagaje [C34], airside [C35] și journey [C36] au propriile intrări și definiții; nu trebuie aplicată automat fiecăruia definiția de „risc” a motorului checkpoint.

6. Dovezi din aeroporturi reale și legătura cu modificările propuse

Sursele sunt precedente externe. Ele nu au fost folosite aici pentru a recalibra automat modelul și nu demonstrează că AeroSynapse a produs rezultatele aeroporturilor citate.

6.1 Capacitate de 3 pasageri/min/linie

Dovada și perioada. Studiul Airport Terminal Facilities Software for Low-Cost Carriers, §3.2.5 [S03], Applied Sciences 16(2), 852, raportează la Pula 3 pasageri/min/linie, pe baza măsurătorilor din iulie 2019

Legătura cu proiectul. Valoarea 180 pasageri/oră/linie din demo are un precedent măsurat într-un context concret

Limita concluziei. Nu dovedește rata la Craiova, rata per angajat sau distribuția constantă de 20 secunde din demo. Sursa este identificată în această documentare; nu pretindem că a fost sursa inițială a constantei din cod

6.2 Mai mult personal la securitate

Dovada și perioada. Schiphol, comunicat din 8 septembrie 2023 [S04]: în perioada 7 iulie-3 septembrie, 95% au trecut de control în 8 minute, așteptarea medie a fost 3 minute, iar 1.050 colegi noi au început să lucreze la securitate

Legătura cu proiectul. Capacitatea de personal este o pârghie operațională reală; operatorul o asociază cu performanța observată

Limita concluziei. Nu izolează efectul cauzal al personalului de pregătirea pasagerilor și alte măsuri; nu furnizează un procent transferabil demo-ului

6.3 Eșalonarea sosirilor prin intervale rezervate

Dovada și perioada. Același comunicat Schiphol [S04] raportează peste 668.000 intervale rezervate până atunci

Legătura cu proiectul. Distribuția în timp a sosirilor poate fi influențată printr-un serviciu folosit efectiv

Limita concluziei. AeroSynapse permite ipoteze despre sosiri; acest lucru nu înseamnă că implementează deja sistemul de rezervare Schiphol sau măsoară respectarea programărilor

6.4 Capacitate mai mare prin echipamente și organizare

Dovada și perioada. Sydney Airport, modernizarea securității T1 [S05]: 15 linii CT noi, proiect finalizat în decembrie 2025; operatorul anunță creșterea capacității de la 4.500 la 5.850 pasageri/oră

Legătura cu proiectul. Schimbarea capacității de procesare și reorganizarea zonelor au un corespondent fizic; raportul anunțat este +30%

Limita concluziei. Este capacitate anunțată a proiectului, nu o măsurare înainte/după a timpului mediu și nici promisiunea că un singur operator produce +30%

6.5 Replanificare și coordonare porți/standuri

Dovada și perioada. EUROCONTROL, A-CDM impact assessment, 18 aprilie 2016 [S06]: evaluare bazată pe 17 aeroporturi cu A-CDM implementat; raportează reducerea timpilor de rulare și a schimbărilor târzii de porți/standuri și utilizarea mai bună a resurselor de handling

Legătura cu proiectul. Partajarea informațiilor și replanificarea coordonată sunt intervenții operaționale documentate, relevante pentru modulele journey/airside

Limita concluziei. Nu validează algoritmul CP-SAT local, procentele sale sau compatibilitatea produsului cu A-CDM

6.6 Performanță de securitate raportată public

Dovada și perioada. Heathrow, comunicat din 13 ianuarie 2025 [S07]: pentru 2024, 92% dintre pasageri au trecut de securitate în sub 5 minute. Pagina de performanță [S08] explică raportarea cozilor în cadrul reglementării CAA

Legătura cu proiectul. Există indicatori publicați cu care putem defini obiective și compara ulterior o cohortă compatibilă

Limita concluziei. „92% sub 5 minute” nu înseamnă medie de 5 minute și nu stabilește P95. Trebuie verificată aceeași definiție a începutului/sfârșitului măsurării

6.7 Extindere fizică la Craiova

Dovada și perioada. Constructorul Construcții Erbașu, proiect finalizat [S09] descrie terminalul operațional, 31.715 m², trei coridoare mobile de îmbarcare și capacitate proiectată de până la 1.933.186 pasageri/an

Legătura cu proiectul. Demonstrează un proiect fizic finalizat și existența unor modificări de infrastructură la aeroportul relevant local

Limita concluziei. Capacitatea anuală proiectată nu este trafic efectiv, capacitate pe minut la securitate sau număr disponibil de angajați; nu se împarte uniform pe minutele anului pentru a calibra cozile

Pentru 2 pasageri/minut la linia mai lentă, 120 secunde la check-in, timpii de mers inițiali, proporțiile de bagaje și coeficienții energetici nu este documentată în codul verificat o proveniență din măsurători aeroportuare. Rămân ipoteze chiar dacă valorile par plauzibile. O referință care confirmă 3 pasageri/minut la Pula nu justifică automat restul parametrilor.

7. Cum se verifică dovezile tehnice

Verificările existente pot fi rulate din directorul intelligence, cu dependențele proiectului:

```
cd intelligence
python -m pytest -q \
  tests/test_passengers.py \
  tests/test_processes.py \
  tests/test_kpis.py \
  tests/test_staffing.py \
  tests/test_provenance.py \
  tests/test_navigation.py \
  tests/test_events.py \
  tests/test_shifts.py \
  tests/test_check_in.py \
  tests/test_energy.py
```

Aceste teste verifică reproductibilitatea populației, limitele distribuțiilor, un exemplu de coadă calculabil manual, excluderea serviciilor neterminate din P95, reconcilierea populației, limitele resurselor, restricțiile de personal/ture, închiderile și calculul energiei. Sunt dovezi de implementare și consistență, nu o validare pe aeroport real. Raportul generat pentru verificarea locală se păstrează în artifacts/data-evidence/, conform convenției repository-ului; nu este o suită completă de integrare a produsului.

Pentru o valoare afișată într-un run concret trebuie păstrate: runId, revizia scenariului, snapshot-ul integral al intrărilor, ID-urile zborurilor, layout-ul, seed-ul, motorul/build-ul, unitatea, minutul/intervalul și populația folosită. Apoi se exportă timestamp-urile individuale sau snapshot-urile KPI și se recalculează formula. provenance.py [C37] definește fingerprint-urile; build-ul include sursele Python și versiunile dependențelor numerice. Fingerprint-urile permit verificarea identității intrărilor și implementării, fără să certifice originea externă.

8. Ce lipsește pentru a susține un beneficiu la un aeroport anume

Parametru / concluzie	Dovada necesară
Cerere și vârfuri reale	Orar datat, pasageri efectivi pe zbor, numărători și timestamp-uri de sosire pe intervale
Capacitate și durate	Timestamp-uri sosire/început/final serviciu, linii efectiv deschise, echipamente, configurația echipei, verificări suplimentare și indisponibilități
Realocare de personal posibilă	Planul liniilor instalate, programul și disponibilitatea oamenilor, calificări/roluri și timpul de transfer
Mers și rerutare	Graf validat al zonelor accesibile, lungimi/timpi măsurați, restricții de acces și separarea fluxurilor
Beneficiu procentual local	Calibrare pe o perioadă, verificare pe alte zile nefolosite la calibrare și comparație a distribuțiilor de așteptare, throughput-ului și celor neprocesați
Economie energetică/monetară	Puteri măsurate inclusiv standby, contoare pe zone și tarif aplicabil acelei perioade

Afirmația susținută în prezent este: **„Pentru aceste intrări sintetice și aceste restricții, simulatorul calculează efectul unei modificări; există precedente reale pentru tipul de intervenție.”** O afirmație precum „la Craiova vom reduce așteptarea cu X%” necesită date și validare locală suplimentare.

Anexa A. Surse externe

Linkurile sunt active. Data consultării: 31 octombrie 2026. Numerele [Sxx] din text identifică sursa; constatările și limitele sunt explicate în secțiunea 6.

[S01] documentația SimPy

https://simpy.readthedocs.io/en/latest/simpy_intro/shared_resources.html

[S02] documentația oficială CP-SAT

https://developers.google.com/optimization/cp/cp_solver

[S03] Airport Terminal Facilities Software for Low-Cost Carriers, §3.2.5

https://www.mdpi.com/2076-3417/16/2/852

[S04] Schiphol, comunicat din 8 septembrie 2023

https://news.schiphol.com/schiphol-looks-back-on-summer-holiday/

[S05] Sydney Airport, modernizarea securității T1

https://www.sydneyairport.com/corporate/planning-and-projects/faster-security-screening-at-international

[S06] EUROCONTROL, A-CDM impact assessment, 18 aprilie 2016

https://www.eurocontrol.int/publication/airport-collaborative-decision-making-cdm-impact-assessment

[S07] Heathrow, comunicat din 13 ianuarie 2025

https://mediacentre.heathrow.com/pressrelease/detail/21683

[S08] Pagina de performanță

https://www.heathrow.com/company/about-heathrow/performance/airport-operations

[S09] Constructorul Construcții Erbașu, proiect finalizat

https://www.erbasu.ro/info-finalizare-lucrare-craiova/

Anexa B. Referințe la cod

Căile de mai jos sunt relative la rădăcina repository-ului AeroSynapse AI. Marcajul L indică linia verificată în commit-ul analizat. Referințele [Cxx] din text trimit la această anexă; pentru verificare pe alt calculator se folosește același checkout al repository-ului.

[C01] SyntheticScenarioSeeder.cs
backend/src/AeroSynapse.Infrastructure/Storage/SyntheticScenarioSeeder.cs : L20

[C02] studio.models.ts
AeroSynapse.Client/src/app/features/scenario-studio/studio.models.ts : L61

[C03] generator.py
intelligence/app/passengers/generator.py : L49

[C04] processes.py
intelligence/app/simulation/processes.py : L182

[C05] metrics.py
intelligence/app/simulation/metrics.py : L19

[C06] staffing.py
intelligence/app/optimization/staffing.py : L65

[C07] shifts.py
intelligence/app/optimization/shifts.py : L57

[C08] forecast.py
intelligence/app/simulation/forecast.py : L13

[C09] dataset.py
intelligence/app/forecast/dataset.py : L23

[C10] model.py
intelligence/app/forecast/model.py : L103

[C11] energy/engine.py
intelligence/app/energy/engine.py : L16

[C12] studio.models.ts
AeroSynapse.Client/src/app/features/scenario-studio/studio.models.ts : L104

[C13] configurarea profilurilor
AeroSynapse.Client/src/app/features/scenario-studio/studio.models.ts : L235

[C14] conversia în timp
intelligence/app/passengers/generator.py : L113

[C15] generator.py:16
intelligence/app/passengers/generator.py : L16

[C16] processes.py:63
intelligence/app/simulation/processes.py : L63

[C17] generator.py:133
intelligence/app/passengers/generator.py : L133

[C18] NavigationGraph.search
intelligence/app/navigation/graph.py : L43

[C19] metrics.py:53
intelligence/app/simulation/metrics.py : L53

[C20] urmărirea termenului
intelligence/app/simulation/processes.py : L151

[C21] kpi-request.json
contracts/kpis/v1/examples/kpi-request.json

[C22] Testul existent

intelligence/tests/test_kpis.py : L21

[C23] StaffingEvaluationReader.cs:110

backend/src/AeroSynapse.Infrastructure/Execution/StaffingEvaluationReader.cs : L110

[C24] StaffingOperations.cs:78

backend/src/AeroSynapse.Application/Management/StaffingOperations.cs : L78

[C25] staffing-request.json

contracts/staffing/v1/examples/staffing-request.json

[C26] Testele solverului

intelligence/tests/test_staffing.py : L37

[C27] StaffingEvaluationWorkflow.cs:66

backend/src/AeroSynapse.Infrastructure/Execution/StaffingEvaluationWorkflow.cs : L66

[C28] StaffingEvaluationPolicy.Assess

backend/src/AeroSynapse.Application/Management/StaffingEvaluation.cs : L48

[C29] percentChange

AeroSynapse.Client/src/app/shared/ui/outcome-cards.ts : L29

[C30] RunComparisonPolicy.cs

backend/src/AeroSynapse.Application/Management/RunComparisonPolicy.cs : L22

[C31] open_lanes

intelligence/app/passengers/generator.py : L161

[C32] testul repartizării

intelligence/tests/test_passengers.py : L282

[C33] forecast.py:124

intelligence/app/simulation/forecast.py : L124

[C34] bagaje

intelligence/app/baggage/engine.py : L13

[C35] airside

intelligence/app/airside/engine.py : L146

[C36] journey

intelligence/app/simulation/journeys.py : L79

[C37] provenance.py

intelligence/app/simulation/provenance.py : L15